Střední průmyslová škola a Vyšší odborná škola, Liberec, příspěvková organizace

VĚTNÁ DRÁHA – APLIKACE PRO ANDROID

Ročníková práce

Autor	**Jan Kočnar**
Obor	**Informační technologie**
Vedoucí práce	**Ing. Tomáš Kazda, DiS.**
Školní rok	**2024/2025**
Počet stran	**26**
Počet slov	**5021**

Anotace

Práce se zaměřuje na vývoj mobilní aplikace pro platformu Android, která má za cíl výuku a procvičování určování větných členů u žáků 6. a 7. třídy základních škol. Aplikace využívá algoritmus založený na binárním stromu pro výuku větných členů a nabízí tři hlavní funkce: režim výuky, režim procvičování a režim testu. Cvičení jsou založena na herních mechanikách a aplikace obsahuje prvky gamifikace. Testy, s vyhodnocováním výsledků, mohou být vytvářeny učiteli. Součástí projektu je také systém pro snadné vkládání, úpravu a rozšiřování testovacích vět a frází.

Summary

The thesis focuses on the development of a mobile application for the Android platform, which aims to teach and practice the determination of sentence elements to students in the 6th and 7th grade of primary schools. The application uses a binary tree-based algorithm for teaching sentence elements and offers three main features: a learning mode, a practice mode and a test mode. The exercises are based on game mechanics and the app includes gamification features. Tests can be created by teachers and provide evaluation of results. The project also includes a system for easy insertion, editing and expansion of test sentences and phrases.

Čestné prohlášení

Prohlašuji, že jsem tuto práci vypracoval samostatně. V práci jsem použil pouze zdroje uvedené v seznamu literatury a všechny převzaté části, včetně textů, obrázků, kódů a výstupů umělé inteligence, jsou řádně označeny a citovány.

V Liberci dne 28.05.2025 .....

Jan Kočnar

Obsah

Úvod ... 1
1 Existující aplikace a jejich limity ... 2
1.1 Umíme česky ... 2
1.2 Moje čeština ... 2
1.3 Pravopisně.cz ... 3
2 Cíle práce ... 4
2.1 Funkční požadavky ... 4
2.1.1 Procvičovací režim ... 4
2.1.2 Testový režim ... 4
2.1.3 Správa obsahu ... 4
2.2 Nefunkční požadavky ... 5
2.2.1 Dostupnost ... 5
2.2.2 Zabezpečení a soukromí ... 5
2.2.3 Použitelnost a přístupnost ... 5
2.2.4 Údržba a nasazení ... 5
2.3 Mobilní aplikace ... 6
2.3.1 Návrh UI a UX ... 6
2.3.2 Gamifikace ... 6
2.3.3 Implementace výukových mechanik ... 6
2.3.4 Implementace testových úloh ... 7
2.4 Správa obsahu ... 7
3 Plán a postup řešení ... 8
3.1 Backend ... 8
3.2 Mobilní aplikace ... 8
3.3 Testování ... 9
3.3.1 Testování backendu ... 9
3.3.2 Unit Testy ... 9
3.3.3 Manuální testování ... 9
3.3.4 Uživatelské testování ... 9
4 Realizace ... 10
4.1 Vývoj backendu ... 10
4.1.1 Databázové schéma ... 10

4.1.2 Uživatelská autentifikace a správa účtů ... 10
4.1.3 Nasazení backendu ... 11
4.1.4 Testování backendu ... 11
4.2 Návrh uživatelského rozhraní ... 12
4.2.1 Návrh hlavních obrazovek ... 12
4.3 Vývoj front-endu ... 14
4.3.1 Struktura aplikace ... 14
4.3.2 Komunikace s backendem ... 15
4.3.3 Aktualizace výukových dat ... 15
4.3.4 Implementace her ... 15
4.3.5 Implementace testových úloh ... 17
4.3.6 Implementace systému nejčastějších chyb ... 17
4.3.7 Unit testy pro kritické funkce ... 18
4.3.8 Stylování a responzivita ... 19
4.3.9 Personalizace ... 19
4.3.10 Nasazení a testování na zařízeních ... 19
4.4 Správa a rozšiřování obsahu ... 19
4.5 Testování a ladění aplikace ... 20
4.6 Publikace aplikace ... 20
Závěr ... 22
Seznam zkratek a odborných výrazů ... 23
Seznam obrázků ... 25
Použité zdroje ... 26
A. Seznam přiložených souborů ... I

Úvod

Výuka českého jazyka na základních školách je nezbytnou součástí vzdělávacího procesu, přičemž jednou z klíčových oblastí je správné určování větných členů. Tento jev představuje pro mnoho žáků 6. a 7. tříd značnou výzvu, která vyžaduje systematické a časté procvičování. Zároveň je stále složitější zaujmout pozornost této věkové skupiny.

V současnosti existuje několik online nástrojů a webových aplikací zaměřených na gramatickou výuku, avšak nedostatek mobilních aplikací se specializací na větné členy omezuje možnosti využití moderních technologií ve výuce.

Cílem této práce je vytvořit mobilní aplikaci pro platformu Android, která bude sloužit k interaktivnímu procvičování určování větných členů formou herních úloh s integrovanou nápovědou. Součástí projektu je také návrh a vytvoření systému pro správu testovacího obsahu, který umožní snadné doplňování a aktualizaci výukových materiálů.

Tato práce by měla přispět k modernizaci výuky českého jazyka a poskytnout žákům i učitelům efektivní nástroj podporující aktivní a zábavné učení. Dále poslouží jako základ pro případné rozšíření a další vývoj aplikace v oblasti digitálního vzdělávání.

1 Existující aplikace a jejich limity

Na trhu existuje několik webových aplikací zaměřených na výuku a procvičování větných členů, ale žádná z nich nenabízí nativní aplikaci určenou přímo pro mobilní platformu Android. Dostupné nástroje bývají často statické, bez možnosti automatického přizpůsobení úrovně znalostí uživatele, moderního vzhledu a ovládání. Chybí jim podpora moderních výukových metod, jako je gamifikace. A žádná z nejznámějších platforem nenabízí žádné nástroje pro učitele, jako je například tvorba a následné vyhodnocení testů.

1.1 Umíme česky

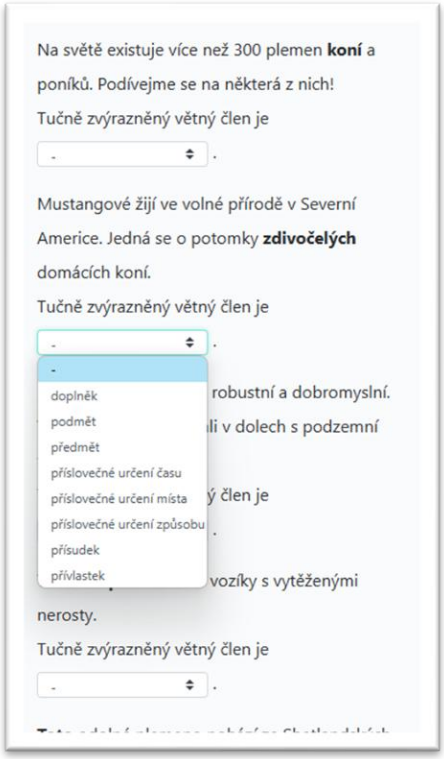
Webový portál Umíme česky je jednou z nejrozšířenějších online platforem pro výuku českého jazyka. Nabízí široké spektrum cvičení zaměřených na gramatiku, pravopis, slovní zásobu i větnou stavbu. Uživatelé si mohou volit mezi různými úrovněmi obtížnosti a získávat okamžitou zpětnou vazbu ke svým odpovědím. V oblasti určování větných členů však portál nabízí jen omezené množství vět a typů úloh.

Platforma funguje výhradně jako webová aplikace a neexistuje její plnohodnotná mobilní verze. Uživatelské rozhraní je z hlediska moderních standardů částečně zastaralé a některé ovládací prvky působí neintuitivně.

1.2 Moje čeština

Webová stránka Moje čeština slouží jako rozsáhlý jazykový portál, který poskytuje teoretické výklady české gramatiky, přehledy jazykových jevů a základní procvičování prostřednictvím jednoduchých testů.

V oblasti větných členů portál nabízí především teoretické vysvětlení s několika málo doprovodnými cvičeními, které mají velmi nepřehledné a nepraktické ovládání a grafické rozhraní odpovídá spíše informačnímu webu než výukové aplikaci.



Obrázek 1 - Procvičování na stránce Moje Čeština

1.3 Pravopisně.cz

Webová platforma Pravopisně.cz představuje z uvedených příkladů nejpropracovanější variantu. Nabízí rozsáhlou sadu cvičení, diktátů a výkladových materiálů zaměřených na český pravopis, gramatiku i stylistiku. Obsah je tematicky členěn a přizpůsoben různým úrovním pokročilosti, čímž se stává vhodným nástrojem nejen pro žáky základních, ale i středních škol.

V oblasti větné stavby a určování větných členů platforma rovněž nezaostává. Nabízí rozsáhlou databázi úloh, které jsou vhodně strukturovány a doplněny o okamžitou zpětnou vazbu. Uživatelské rozhraní je intuitivní a cvičení působí oproti ostatním zmíněným řešením interaktivněji a přístupněji.

Přes své kvality však platforma postrádá některé pokročilé funkce, například učitelský režim umožňující tvorbu vlastních testů. Dalším omezením je limitovaný denní přístup, bez placeného předplatného může uživatel absolvovat pouze jedno cvičení denně.

2 Cíle práce

Cílem této práce je navrhnout a vytvořit výukovou mobilní aplikaci pro platformu Android, která bude sloužit k procvičování a upevňování znalostí větných členů. Aplikace je určena především pro žáky 6. a 7. tříd základních škol, a to formou interaktivních cvičení s principy gamifikace a herních mechanik.

Téma a zadání této práce bylo navrženo Romanou Wagnerovou, která vytvořila algoritmus pro určování větných členů a podílí se na správě sad vět potřebných pro tuto aplikaci. Zadání této práce je také částí její disertační práce. Některé požadavky, funkční i nefunkční, tedy vychází z externího zadání.

2.1 Funkční požadavky

Funkční požadavky definují, jaké konkrétní funkce musí výsledná aplikace uživatelům poskytovat.

2.1.1 Procvičovací režim

- Možnost výběru vět k procvičování dle konkrétního typu větného členu nebo skupiny větných členů.
- Možnost procvičovat věty, ve kterých uživatel v minulosti chyboval.
- Zobrazení nápovědy při procvičování.
- Zobrazení evaluace po dokončení procvičování.
- Možnost postupovat v přívětivém a gamifikovaném systému.

2.1.2 Testový režim

- Možnost vytvoření a spuštění online testu pro celou třídu v reálném čase.
- Tvorba testu s vlastními parametry.
- Zobrazení individuálních výsledků žáků i souhrnných statistik pouze pro tvůrce testu.
- Přístup k historii testů pouze jejich autorem.
- Personalizace pro žáky ve formě avatara a jména.

2.1.3 Správa obsahu

- Možnost aktualizace výukového obsahu bez nutnosti aktualizovat celou aplikaci.
- Možnost změny adresy backendového serveru bez zásahu do kódu aplikace.

2.2 Nefunkční požadavky

Nefunkční požadavky určují vlastnosti systému, které se netýkají přímo jeho funkcionality, ale ovlivňují jeho použitelnost, výkonnost, dostupnost a bezpečnost.

2.2.1 Dostupnost

- Doba odezvy serveru musí být dostatečně nízká, aby bylo zajištěno rychlé zpracování požadavků a příjemný uživatelský zážitek.

2.2.2 Zabezpečení a soukromí

- Přístup k výsledkům testů musí být omezen pouze na jejich autora.
- Musí být zajištěna autentizace a autorizace uživatelů tak, aby nebylo možné upravovat cizí údaje.
- Aplikace nesmí umožňovat žádným způsobem manipulaci s výsledky testů nebo identitou jiného žáka.
- Veškerá komunikace mezi aplikací a serverem musí být šifrována.
- Aplikace nesmí shromažďovat ani zpracovávat zbytečná osobní data žáků.

2.2.3 Použitelnost a přístupnost

- Uživatelské rozhraní musí být přehledné a intuitivní i pro nové uživatele.
- Aplikace musí být schopná fungovat v omezeném režimu i bez připojení k internetu.
- Při návrhu musí být zohledněna různá rozlišení obrazovek, včetně tabletů.
- Aplikace musí mít plnou funkčnost bez nutnosti zakládat účet jak žákem, tak i učitelem.

2.2.4 Údržba a nasazení

- Backendové API i mobilní aplikace musí být jednoduše nasaditelné.
- Kritické části systému (např. generování testových úloh) musí být pokryty unit testy.
- Musí být zaveden mechanismus pro hlášení chyb ze strany uživatelů.

2.3 Mobilní aplikace

Jedním z hlavních cílů této práce je návrh a realizace funkční, intuitivní a uživatelsky přívětivé mobilní aplikace určené pro platformu Android.

2.3.1 Návrh UI a UX

Při návrhu uživatelského rozhraní (UI) a uživatelské zkušenosti (UX) musí být kladen důraz na jednoduchost, přehlednost a intuitivní ovládání. Grafický styl aplikace bude navržen tak, aby byl vizuálně atraktivní, avšak nerušivý při učení. Použitá typografie a rozvržení prvků musí reflektovat potřebu maximální srozumitelnosti. Barevná paleta větných členů by měla vycházet z barev běžných ve školním prostředí při výuce tohoto tématu.

2.3.2 Gamifikace

Gamifikace bude v aplikaci využita jako klíčový motivační prvek, který by měl zvýšit zapojení uživatelů do výuky. Do aplikace budou integrovány herní mechaniky, které propojí učení se zábavou a poskytnou okamžitou zpětnou vazbu.

Hlavním prvkem bude systém tzv. arén, inspirovaný strukturou známou z herního prostředí (např. Clash Royale). Každá aréna reprezentuje úroveň obtížnosti. Postup do vyšší arény bude podmíněn úspěšným dokončením všech úloh v aréně předchozí, čímž se podporuje systematické a motivované procvičování.

2.3.3 Implementace výukových mechanik

Výuka v aplikaci bude probíhat formou interaktivního procvičování sloužícího k osvojení a upevnění znalostí o větných členech. Uživatel bude mít při řešení úloh možnost zobrazit nápovědu, která mu pomůže lépe pochopit problematiku a správně určit jednotlivé větné členy.

Procvičování bude probíhat formou ucelené hry skládající se z několika navazujících cvičení. K dispozici bude několik typů úloh lišících se formou zadání a úrovní obtížnosti. Po jejich dokončení bude následovat závěrečná evaluace, jež uživateli poskytne přehled o jeho výsledcích, jako je celková úspěšnost. Hráč musí mít možnost vybrat si, jaké větné členy chce zrovna procvičovat.

Pro cílené procvičování bude implementován systém nejčastějších chyb, který bude sledovat věty, které jsou pro uživatele obtížné a nabídne jim možnost procvičit si právě ty.

2.3.4 Implementace testových úloh

Testový režim tvoří samostatnou část aplikace určenou ke kontrolnímu ověření znalostí. Učitel bude mít možnost vytvářet vlastní testy s definovanými parametry, jako je počet otázek, tematické zaměření nebo časový limit. Výsledky testů jsou následně zpracovány a budou zobrazovány v reálném čase autorovi testu. Učitel tak získá možnost sledovat jak individuální úspěšnost žáků, tak statistiky celé třídy.

2.4 Správa obsahu

Součástí práce je administrační rozhraní umožňující správu výukového obsahu bez nutnosti zásahů do zdrojového kódu. Administrační panel musí umožňovat nahrání nových dat a jejich zobrazení. Sada vět se musí aktualizovat v aplikaci hned, jakmile to bude možné.

3 Plán a postup řešení

Realizace aplikace byla rozdělena do několika samostatných fází zahrnujících návrh, implementaci, testování a nasazení jednotlivých částí systému. Výběr technologií byl podřízen požadavku na nízkou hardwarovou náročnost, jednoduché nasazení a minimální údržbu.

3.1 Backend

Kvůli vzdálené správě obsahu a online testovým úlohám je nutné navrhnout a vytvořit backend a to včetně databáze a API. Backend je navržen jako co nejvíce odlehčený, aby minimalizoval nároky na serverový provoz. Na server jsou proto ukládána pouze ta data, která musí být sdílena mezi více uživateli, nebo vyžadují centralizovanou správu. Patří sem například testy vytvářené učiteli, výsledky testování nebo data sloužící k procvičování a testům. Ostatní data jsou uchovávána přímo v aplikaci, aby se snížila závislost na internetovém připojení a zrychlil se běh aplikace v běžných podmínkách.

Pro implementaci serverového rozhraní byl zvolen framework Express.js, běžící v prostředí Node.js. Jedná se o technologii umožňující rychlou tvorbu, správu a následné nasazení API endpointů.

Další důležitou částí je databázová vrstva, ta bude implementována pomocí SQLite, lehké a přenosné databáze, která je uložena jako jeden soubor, což přispívá k jednoduchému nasazení, jelikož není nutný samostatný databázový server.

Datový model a přístup k databázi je spravován pomocí ORM nástroje Prisma, který zjednodušuje práci s databází a umožňuje typově bezpečný přístup ke strukturám dat.

Součástí backendu bude i administrační prostředí, pro které se použije moderní javascriptový framework Svelte.

3.2 Mobilní aplikace

Pro návrh uživatelského rozhraní bude použit nástroj Figma, který umožňuje efektivní prototypování, sdílení návrhů a spolupráci.

Pro vývoj mobilní aplikace byl zvolen framework React Native, který umožňuje tvorbu aplikací pro Android i iOS z jednoho zdrojového kódu, což zajišťuje konzistentní uživatelský zážitek napříč platformami. Pro zjednodušení prototypování byla do projektu přidána knihovna Expo, která díky své mobilní aplikaci Expo Go umožňuje rychlé a příjemné testovaní aplikace, a to i bez fyzicky připojeného mobilního zařízení.

Jazykem vývoje je TypeScript, který zvyšuje typovou bezpečnost a předvídatelnost kódu.

3.3 Testování

Testování aplikace proběhne ve více fázích, které budou zahrnovat automatizované testy i manuální ověřování funkcionality nejen na cílovém zařízení.

3.3.1 Testování backendu

Nástrojem pro testování serverového rozhraní se stane Postman, který umožní systematické ověřování jednotlivých endpointů API. Každý důležitý požadavek bude otestován nejen na správnou odpověď v ideálním případě, ale také na chování při různých typech chyb jako jsou chybějící parametry, neautorizovaný přístup nebo neexistující záznamy.

3.3.2 Unit Testy

Pro kritické části kódu, zejména generování výukových a testových dat, budou implementovány unit testy pomocí frameworku Jest. Automatizované testování umožní včas odhalit chyby při změnách v kódu, snížit počet opakovaných chyb a zrychlí vývoj. Testy se zaměří zejména na části aplikace, kde se provádějí složitější datové operace nebo kde by chyba mohla vést k chybnému vyhodnocení výsledků.

3.3.3 Manuální testování

Aplikace bude opakovaně testována ručně přímo na zařízení autora s operačním systémem Android, a to během téměř všech fází vývoje. Tímto způsobem by mělo být možné ověřit nejen správné technické fungování aplikace, ale i uživatelský zážitek jako plynulost ovládání, čitelnost textu, srozumitelnost rozhraní a vizuální nedostatky.

3.3.4 Uživatelské testování

V závěrečné fázi vývoje bude aplikace předložena vybrané skupině reálných uživatelů – od žáků Střední průmyslové školy v Libereci a rodinných příslušníků. V ideálním případě toto testování odhalí jen drobné nedostatky, které půjdou opravit bez větších zásahů do vzhledu či logiky aplikace

4 Realizace

Tato kapitola popisuje samotnou realizaci projektu. Vychází z předchozího návrhu a plánu a popisuje, jak byly jednotlivé části systému vytvořeny, jaké technologie byly použity, proč byly zvoleny právě tyto přístupy, a jak se vyvíjela výsledná podoba aplikace. Realizace je rozdělena do jednotlivých částí podle klíčových komponent systému.

4.1 Vývoj backendu

Pro zajištění správy dat, autentizace hráčů a zpracování herní logiky byla navržena a implementována serverová část aplikace postavená na frameworku Express.js. Tato část systému poskytuje rozhraní ve formě REST API, které umožňuje komunikaci s klientskou aplikací.

Serverová část aplikace poskytuje REST API, které slouží pro komunikaci mezi klientem a databází. Každý endpoint má jasně definovanou funkci, používá odpovídající HTTP metody a vrací data ve formátu JSON.

4.1.1 Databázové schéma

Mezi hlavní tabulky databáze patří:

- **Player** – obsahuje informace o hráčích (jméno, barvy postavy, zvolenou raketu, verzi klienta a tajný klíč pro autentizaci).
- **Game** – uchovává informace o vytvořených hrách (autor, obtížnost, galaxie, typy úloh, počet otázek, náhodný seed atd.).
- **GameSession** – zaznamenává herní relace jednotlivých hráčů (čas začátku, skóre, správné odpovědi, dokončení atd.)

Soubory se sadami vět jsou uloženy mimo databázy ve formě json souborů, toto rozhodnutí bylo učiněno za účelem snížení odezvy při vyšším počtu požadavků.

4.1.2 Uživatelská autentifikace a správa účtů

Aplikace nevyužívá klasické účty s přihlašovacím jménem a heslem. Každému hráči je při vytvoření automaticky přidělen **unikátní identifikátor** a **tajný klíč**.

Tyto hodnoty jsou uchovávány na klientovi a odesílány s každým požadavkem prostřednictvím HTTP hlaviček.

Na druhé straně jsou pak správci obsahu, ty mohou pomocí chráněných endpointů přidávat a upravovat sady vět. Přístup k těmto endpointům je chráněn HTTP Basic autentizací.

4.1.3 Nasazení backendu

Celý backend je připraven k provozu pomocí Docker kontejneru, který zajišťuje jednotné prostředí bez ohledu na cílový server. Backend je v době psaní této práce nasazený na školním sandboxu (https://id-102.pslib.cloud), což je izolované prostředí pro bezpečné spouštění studentských projektů.

Docker kontejner obsahuje vše potřebné – Node.js aplikaci, databázi a nastavení prostředí.

```
FROM oven/bun:latest

WORKDIR /app
COPY . .
RUN apt-get update -y && apt-get install -y openssl
RUN bun install

EXPOSE 5173
EXPOSE 8080
USER bun
RUN ls -la

ENTRYPOINT ["bun", "index.ts"]
```

Docker soubor použitý na sestavení backendu

Pro zjednodušení nasazení byl vytvořen GitHub Actions workflow, který po každém pushnutí změn do repozitáře spustí docker file a nahraje aplikaci na zabezpečený endpoint sandboxu.

4.1.4 Testování backendu

Testování serverové části probíhalo ručně pomocí nástroje Postman, které umožňují snadné ověřování jednotlivých API požadavků.

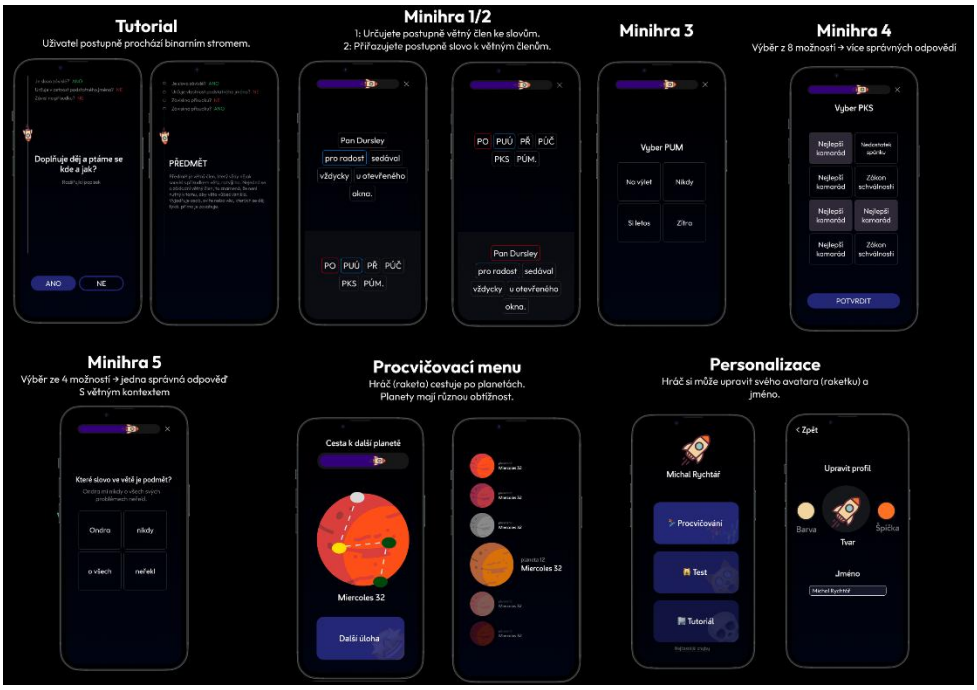
Každý endpoint byl testován na:

- správnost odpovědí (status kódy, obsah JSON)
- validaci vstupních dat
- správné uložení dat do databáze

Dále byly testovány i hraniční případy, jako je připojení ke hře, která již skončila, odeslání neúplné relace, nebo nesprávná kombinace vstupních dat.

4.2 Návrh uživatelského rozhraní

Návrh byl vytvořen pro ujasnění struktury aplikace a vytvoření hrubého rozpoložení jednotlivých částí, vzhled aplikace byl v průběhu vývoje několikrát upravován na základě testování.



Obrázek 2 - Hrubý návrh aplikace

4.2.1 Návrh hlavních obrazovek

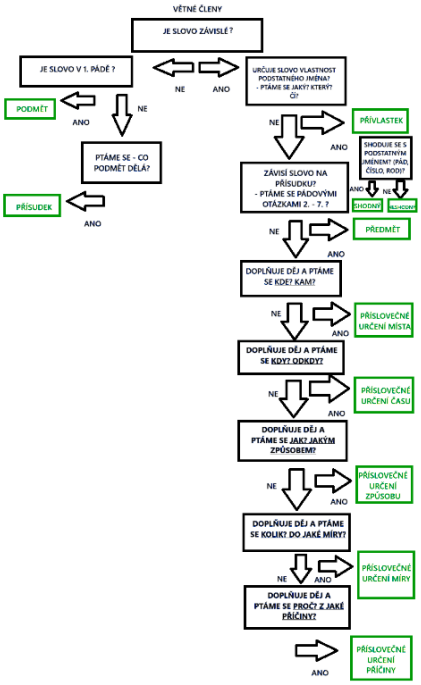
Rozhraní je rozděleno do několika obrazovek, mezi ty nejdůležitější patří: hlavní menu, obrazovka hry, nejčastější chyby, profil, současná aréna a přehled arén. Součástí návrhu bylo také jednoduché nastavení profilu uživatele a možnost sledování pokroku.

4.2.1.1 Aréna

Uživatel si může vybrat, kterou skupinu větných členů chce procvičovat. Má možnost upravit si profil, zobrazit nejčastější chyby a spustit procvičování dané oblasti.

4.2.1.2 Tutoriál

Tutoriál je interaktivní a organizován ve formě binárního stromu, kde je uživateli pokládána otázka s odpovědí „ano" nebo „ne", což usnadňuje pochopení struktury vět a pojmů.

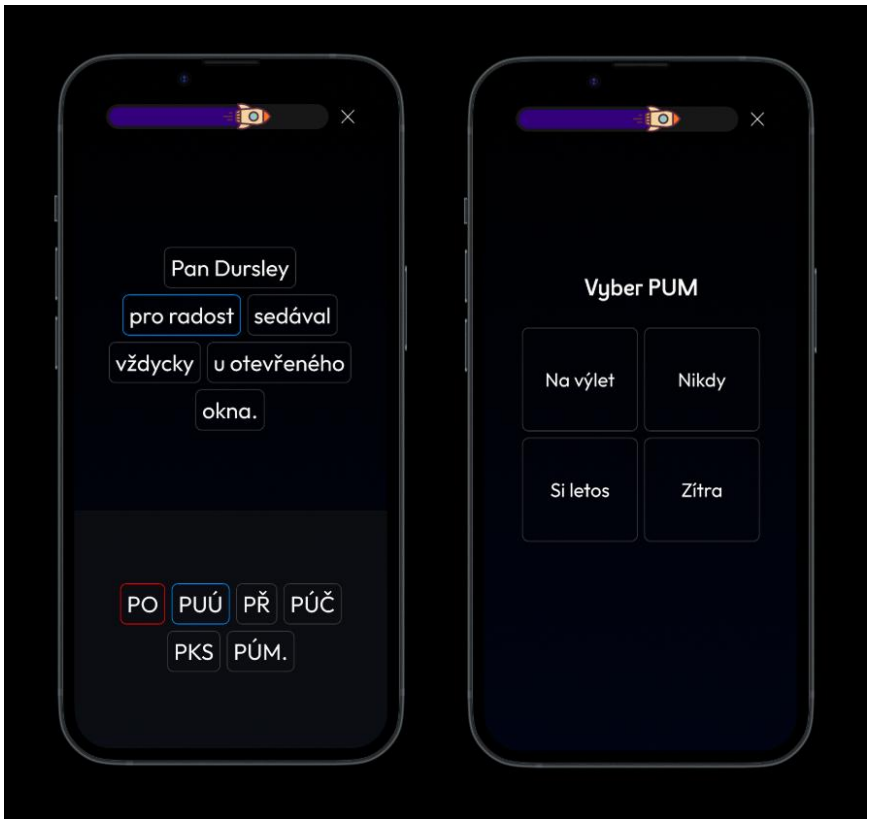


Obrázek 3 - Algoritmus pro určování větných členů v podobě binárního stromu vytvořen Mgr. Romanou Wágnerovou

4.2.1.3 Návrh miniher

Aplikace má 8 různých typů miniher, ale ve Figmě byly navrženy pouze některé z nich, zejména kvůli tomu, že minihry se vzhledem od sebe často liší pouze minimálně.

Z pohledu vzhledu lze minihry rozdělit do dvou hlavních kategorií - určování větných členů ve větě (typ MARK) a výběr větných členů z nabídky (typ SELECT), každá kategorie má 4 varianty.



Obrázek 4 - Návrh miniher typu MARK (vlevo) a SELECT (vpravo)

4.2.1.4 Testování návrhu

Návrh byl průběžně testován v oficiální aplikaci, kterou Figma za tímto účelem nabízí, na základě informací byl návrh postupně dolazován.

4.3 Vývoj front-endu

React Native projekt byl založen z Expo šablony pomocí příkazu

```
npx create-expo-app@latest
```

4.3.1 Struktura aplikace

Aplikace je rozdělena na komponenty, které jsou modularizovány podle funkcí. Navigace byla implementována pomocí React Navigation. Hlavní obrazovky (např. Aréna, Test, Tutoriál) jsou dostupné skrze spodní navigaci, zatímco všechny ostatní obrazovky (např. nastavení profilu, výsledky testů) jsou řešeny Stack Navigátorem.

Komponenty byly navrženy modulárně a přehledně, což umožňuje jejich opětovné využití v různých částech aplikace a zjednodušuje údržbu. Logika byla oddělena od UI prvků, čímž se podařilo dodržet zásady čistého a dobře čitelného kódu.

Pro řízení stavu aplikace bylo využito nástroje React Context, který umožňuje sdílet stav mezi jednotlivými komponentami bez nutnosti předávání dat přes props. Kontext byl využit například pro uchovávání informací o přihlášeném uživateli, jeho postupu ve hře, aktuálním režimu aplikace a načtených datech.

4.3.2 Komunikace s backendem

Pro komunikaci aplikace se serverem byl vytvořen React Hook useApi, a to hlavně kvůli konfiguračnímu souboru uloženém na GitHub Pages a možnosti záložního serveru. Aplikace se po spuštění pokusí stáhnout konfigurační soubor z GitHubu a pokud selže, použije výchozí konfiguraci. Po načtení konfigurace se aplikace spojí s prvním serverem, který je zde uvedený, pokud se spojení nepodaří navázat, udělá to stejné se záložním serverem. Tímto způsobem je možné nejen okamžitě změnit URL serveru, ale také lze při migraci nastavit původní server jako zálohu a zaručit tak stálou funkcionalitu i v případě potíží na novém serveru.

4.3.3 Aktualizace výukových dat

Při každém spuštění aplikace, nebo když se telefon připojí na internet, se zkontroluje verze stažených a dat a v případě zastaralé verze se data aktualizují. Dále se zapíšou jako json soubor do perzistentního úložiště a aktualizuje se poslední stažená verze. V případě, že internet není k dispozici se načítají poslední uložená data, pokud ani ta nejsou k dispozici, použije se výchozí sada vět, která je distribuována s aplikací samotnou. Tímto způsobem má uživatel k dispozici co nejnovější data i v moment, kdy zrovna není připojen k internetu.

4.3.4 Implementace her

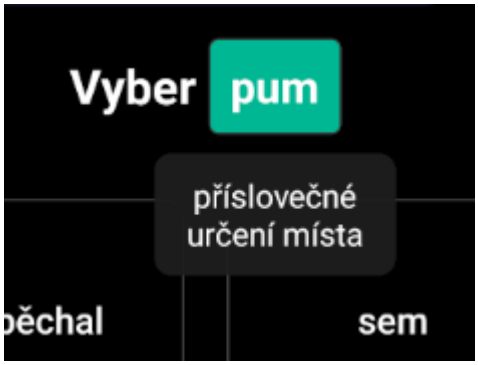
Všechny hry jsou implementovány pomocí jedné obrazovky a hra určuje pouze část jejího vnitřního obsahu, tento způsob zjednodušuje přidávání animací a zrychluje načítání her oproti možnosti, kde má každá hra svoji obrazovku.

Data pro hry jsou generována na základě hodnoty "seed", což zaručuje opakovatelnost výsledků a možnost synchronizace. Po dokončení hry je zobrazen evaluační obrazovka, kde může uživatel postoupit do dalšího levelu nebo si procvičování zopakovat v případě nedostatečné úspěšnosti.

V aplikaci je celkem 8 typů různých úloh, všechny typy jsou rozepsané v následující tabulce. Hry, které začínají slovem „Mark“ mají nabídku větných členů, ty, které začínají slovem „Select“ mají mřížku s možnostmi.

INTERNÍ NÁZEV	POPIS
MARK_WORDS	Určování větných členů vyobrazené věty pomocí nabídky členů, každý člen lze použít pouze jednou a žádný není navíc.
MARK_TYPES	Doplňování slov podle vyobrazených větných členů, každé slovo lze použít právě jednou.
MARK_TYPE_ONE_WORD	Identické jako MARK_WORDS, ale určuje se pouze jeden větný člen.
MARK_WORDS_ALL_TYPES	Identické jako MARK_WORDS, ale v nabídce jsou všechny větné členy a každý lze použít libovolněkrát.
SELECT_MULTIPLE	Výběr více slov najednou podle zadaného větného členu.
SELECT_MULTIPLE_W_SENTENCE	Výběr více slov najednou podle zadaného větného členu z věty.
SELECT_ONE_W_SENTENCE	Výběr jednoho slova z věty podle zadaného větného členu.
SELECT_TYPE	Výběr typu ze zadané věty.

Pro lepší uživatelský zážitek byl do aplikace přidán systém tooltipů – kdykoli je vyzobrazen nějaký větný člen, je možné zobrazit si jeho celý název podržením prstu na něm.



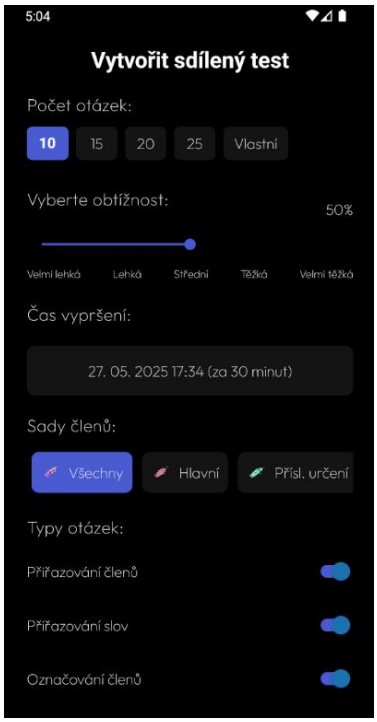
Obrázek 5 - Aktivní tooltip

4.3.5 Implementace testových úloh

Testy fungují velmi podobně jako běžné procvičování, ale s přidanou synchronizací s učitelem, který má rozsáhlou možnost modifikace testu. Může si vybrat počet otázek, obtížnost, jaké typy úloh se budou zobrazovat, jak dlouho bude test platný a jestli mají mít všichni žáci stejné otázky.

Hráč se připojuje za pomocí vygenerovaného kódu, nazpět získá nastavení hry a frontend podle těchto pravidel vygeneruje daný počet úloh.

Během vyplňování testu hráč postupně odesílá odpovědi a skóre prostřednictvím PATCH požadavků. Každá relace je zaznamenávána do databáze, včetně času, výsledků a správných odpovědí.



Obrázek 6 - Tvorba testu z pohledu učitele

4.3.6 Implementace systému nejčastějších chyb

Po každé chybě, kterou uživatel udělá jak v procvičování, tak v testu se chyba zapíše do lokálně uloženého souboru. S chybou se ukládá i počet, kolikrát uživatel chybu udělal a kolikrát ji naopak odpověděl správně. Po třech správných odpovědích se chyba

smaže. Aplikace umožňuje vytvoření testu pouze s nejčastějšími chybami. Každou chybu lze také zobrazit jednotlivě a uživatel se tak může podívat na správné řešení.

Obrázek 7 - Zobrazení nejčastějších chyb

4.3.7 Unit testy pro kritické funkce

Unit testy slouží k ověření funkčnosti kritických částí frontendu. Testy byly spouštěny v průběhu vývoje a sloužily jako nástroj pro rychlou detekci chyb při úpravách klíčové logiky aplikace.

Testování bylo zaměřeno především na generování procvičovacích a testových dat, kde je klíčová správnost výstupů a konzistence logiky. Mezi nejdůležitější testy patří kontrola toho, jestli stejný seed vždy vygeneruje stejné otázky a naopak, zda různý seed vygeneruje různé otázky.

```
it('consistent data with same seed', () => {
  const levels1 = generateRandomGameLevels(3, 123, mockData);
  const levels2 = generateRandomGameLevels(3, 123, mockData);
  expect(levels1).toEqual(levels2);
});
```

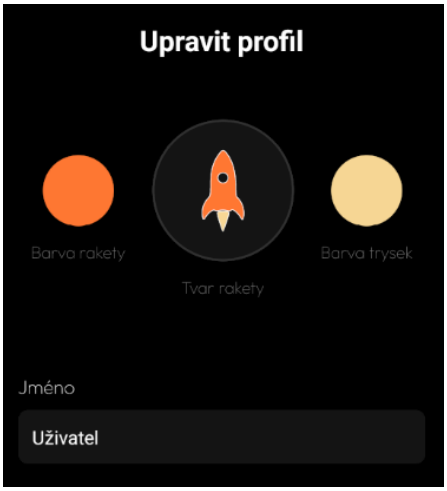
Unit test pro kontrolu generování dat napsaný v frameworku Jest

4.3.8 Stylování a responzivita

Stylování aplikace bylo realizováno pomocí nástroje StyleSheet z React Native. Komponenty byly navrženy tak, aby se přizpůsobily různým velikostem obrazovek a zařízení.

4.3.9 Personalizace

Každý uživatel má možnost upravit si přezdívku a svůj avatar, který se pak vyskytuje napříč aplikací.



Obrázek 9 - Možnosti upravení avataru

Obrázek 8 - Úprava barevné palety rakety

4.3.10 Nasazení a testování na zařízeních

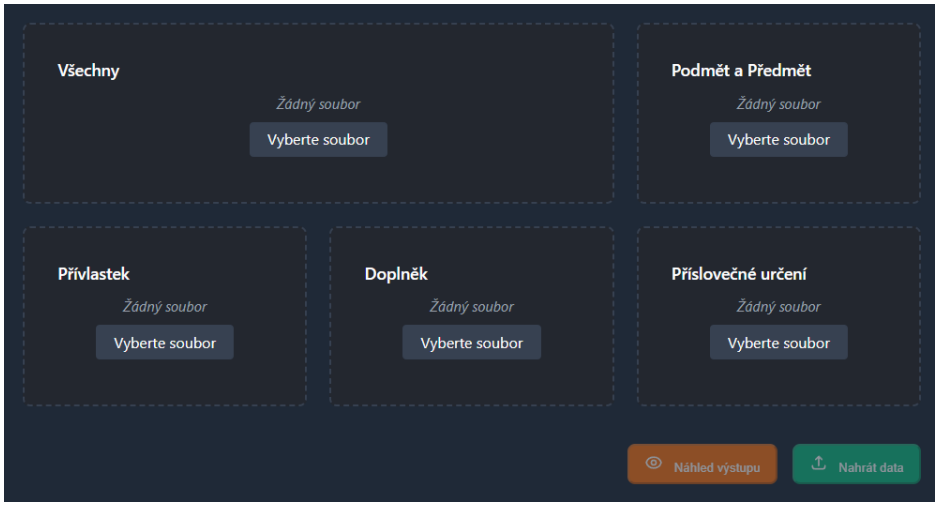
Aplikace byla sestavována pomocí služby Expo EAS pomocí CLI

```
npx eas-cli build -p android --profile preview
```

Výstupem byl .aab soubor, který byl následně konvertován na .apk, pomocí bundle-tools od Googlu, pro čistě testovací účely.

4.4 Správa a rozšiřování obsahu

Administrační panel je dostupný přes jednoduché webové rozhraní dostupné na školním sandboxu. Dají se zde zobrazit aktuální data, která na serveru jsou, nebo nahrát nová pomocí jednoduchého prostředí.



Obrázek 10 - Část administračního panelu pro nahrávání souborů

Data jsou ukládána do souborů ve formátu JSON, což zajišťuje rychlejší čtení oproti SQL dotazu v databázi. V zájmu zajištění jednoduché úpravy se data nahrávají v csv formátu ve velmi jednoduché, standardizované struktuře: slovo, typ

Našel	**př**	**psa**	**pt**	**zmoklého.**	**d**
Vrátil se	př	z výletu	pum	hladový.	d

Po nahrání a validaci struktury tabulky se data přímo na frontendu zparsují do json formátu a pošlou se na zabezpečený endpoint backendu.

4.5 Testování a ladění aplikace

Závěrečná fáze testování probíhala za účasti cílové skupiny. Zpětná vazba byla získávána v několika etapách od studentů Střední průmyslové školy Liberec, rodinných příslušníků vývojářů a dalších dobrovolníků. Díky tomu se podařilo odhalit nedostatky v uživatelském rozhraní, herních mechanikách i výukových prvcích. Tyto poznatky pak byly využity k postupnému a cílenému vylepšování aplikace.

4.6 Publikace aplikace

Pro správu sestav a distribuce na platformě Android byl využit nástroj Expo Application Services (EAS), který umožňuje rychlé sestavování aplikací v cloudu bez nutnosti instalace Android Studia nebo dalších lokálních nástrojů.

Distribuce aplikací na Google Play vyžaduje vlastnictví vývojářského účtu Google Play Developer, jehož zřízení je spojeno s jednorázovým poplatkem ve výši 25 USD.

Vzhledem k tomu, že se jedná o školní projekt, byla aplikace publikována v rámci školního účtu.

Po nastavení aplikace v konzoli Google Play Console bylo možné zahájit testování prostřednictvím uzavřeného nebo otevřeného beta testu, což umožňuje sbírat zpětnou vazbu od uživatelů ještě před finálním zveřejněním. Po úspěšném schválení aplikace ze strany Google je možné ji plně distribuovat prostřednictvím obchodu Google Play.

Závěr

Tato práce se zabývala návrhem, vývojem a publikací mobilní aplikace pro výuku určování větných členů. Během realizace se podařilo splnit všechny stanovené cíle – návrh vhodného a přístupného uživatelského rozhraní, implementace několika různých typů cvičení a publikace aplikace na Google Play, kde není ještě veřejně dostupná, ale je již připravená na testování.

V úvodní fázi byl vytvořen katalog požadavků shrnující očekávané funkce, uživatelské scénáře a technická omezení. Následoval návrh optimální datové struktury, která byla přizpůsobena jak potřebám výuky, tak provozní jednoduchosti. Zvláštní pozornost byla věnována také vkládání nového obsahu a úpravy stávajících dat, a to prostřednictvím čitelného CSV formátu a administračnímu panelu. Uživatelské rozhraní bylo navrženo tak, aby bylo nejvíce jednoduché a srozumitelné, ale zároveň zajímavé pro žáky druhého stupně základní školy.

Aplikace také byla doplněna o prvky personalizace – uživatel může sledovat svůj pokrok, zobrazit si často se opakující chyby a na jejich základě se cíleně zlepšovat. K vyšší motivaci přispívá i zapojení herních prvků: aplikace je koncipována jako série výukových úrovní (arén), přičemž samotné úlohy nabízejí různorodé typy zadání i úrovně obtížnosti. Důležitou součástí systému je i možnost zapojení učitele, který může připravovat vlastní testy a sledovat výsledky svých žáků.

Součástí vývoje bylo také důkladné testování – od automatizovaných jednotkových testů přes ruční testování na cílovém zařízení až po reálné uživatelské testy s vybranými žáky. Získaná zpětná vazba vedla k řadě vylepšení, zejména v oblasti srozumitelnosti a použitelnosti.

Výsledkem je tak plnohodnotná aplikace, která propojuje učení a zábavu. Vzhledem ke zvolené architektuře je možné aplikaci dále rozšiřovat o nové úlohy. Upravovat výukový je možný bez nutnosti zásahů do samotného kódu. Věřím tedy, že aplikace přinese konkrétní přínos nejen pro žáky, ale i pro učitele.

Seznam zkratek a odborných výrazů

API

Application Programming Interface – programové rozhraní aplikace, standardizovaný způsob komunikace mezi různými softwarovými komponentami.

CSS

Cascading Style Sheets – jazyk pro definování vzhledu a formátování webových stránek.

CSV

Comma Separated Values – textový formát pro ukládání strukturovaných dat oddělených čárkami.

EAS

Expo Application Services – cloudová služba pro sestavování a nasazování mobilních aplikací vytvořených pomocí Expo frameworku.

HTTP

HyperText Transfer Protocol – protokol pro přenos dat na internetu, používaný pro komunikaci mezi webovými servery a prohlížeči.

HTTP Basic autentizace

Jednoduchá metoda autentizace v HTTP protokolu, kdy se uživatelské jméno a heslo odesílají zakódované v hlavičce požadavku.

iOS

iPhone Operating System – mobilní operační systém vyvinutý společností Apple pro iPhone a iPad.

JSON

JavaScript Object Notation – lehký formát pro výměnu dat, snadno čitelný jak pro lidi, tak pro počítače.

ORM

Object-Relational Mapping – technika pro práci s databází pomocí objektově orientovaného programování.

REST

Representational State Transfer – architektonický styl pro návrh webových služeb a API.

SQL

Structured Query Language – programovací jazyk pro práci s relačními databázemi.

UI

User Interface – uživatelské rozhraní, způsob interakce mezi uživatelem a aplikací.

URL

Uniform Resource Locator – webová adresa specifikující umístění zdroje na internetu.

UX

User Experience – uživatelská zkušenost, celkový zážitek uživatele při používání aplikace nebo systému.

Backend

Serverová část aplikace běžící na serveru, která zajišťuje logiku aplikace, správu dat, komunikaci s databází a poskytuje API pro komunikaci s klientskými aplikacemi

Frontend

Klientská část aplikace, která běží na zařízení uživatele a zajišťuje uživatelské rozhraní, zobrazování dat a interakci s uživatelem.

Stack Navigátor

Typ navigace v React Native aplikacích, který spravuje obrazovky uspořádané do zásobníku (stack), kde nové obrazovky se přidávají na vrchol a uživatel se může vracet k předchozím obrazovkám pomocí tlačítka zpět.

React Context

Mechanismus v React frameworku pro sdílení dat mezi komponentami bez nutnosti předávání přes props, umožňující efektivní správu globálního stavu aplikace.

Props

Mechanismus v React frameworku pro předávání dat a funkcí z rodičovské komponenty do podřízených komponent, umožňující komunikaci mezi komponentami.

Endpoint

Specifická adresa (URL) v rámci API, která definuje přístupový bod pro komunikaci mezi klientskou aplikací a serverem, umožňující provádění konkrétních operací jako načítání, ukládání nebo mazání dat.

Seznam obrázků

Obrázek 1 - Procvičování na stránce Moje Čeština 3
Obrázek 2 - Hrubý návrh aplikace 12
Obrázek 3 - Algoritmus pro určování větných členů v podobě binárního stromu vytvořen Mgr. Romanou Wágnerovou 13
Obrázek 4 - Návrh miniher typu MARK (vlevo) a SELECT (vpravo) 14
Obrázek 5 - Aktivní tooltip 16
Obrázek 6 - Tvorby testu z pohledu učitele 17
Obrázek 7 - Zobrazení nejčastějších chyb 18
Obrázek 8 - Úprava barevné palety rakety 19
Obrázek 9 - Možnosti upravení avataru 19
Obrázek 10 - Část administračního panelu pro nahrávání souborů 20

Použité zdroje

1. **Refine.** Refine. *React Design Patterns.* [Online] 05. 09 2024. [Citace: 20. 05 2025.] https://refine.dev/blog/react-design-patterns.
2. **kibakdi@gmail.com.** Almax Design Agency. *The Psychology of Light vs Dark Modes in UX Design.* [Online] [Citace: 05. 05 2025.] https://almaxagency.com/design-trends/the-psychology-of-light-vs-dark-modes-in-ux-design/.
3. **Google.** Google. *Vytvoření a nastavení aplikace.* [Online] [Citace: 26. 05 2025.] https://support.google.com/googleplay/android-developer/answer/9859152.
4. **Postman.** Postman. *Test API integrations and data flow in Postman.* [Online] 01. 05 2024. [Citace: 10. 05 2025.] https://learning.postman.com/docs/tests-and-scripts/test-apis/integration-testing/.
5. **DuVander, Adam.** Stoplight. *API Design Patterns for REST.* [Online] 24. 11 2021. [Citace: 13. 05 2025.] https://blog.stoplight.io/api-design-patterns-for-rest-web-services.
6. **P, Jeff.** Medium. *Unit Testing with Jest.* [Online] 30. 10 2023. [Citace: 23. 05 2025.] https://waxlyrical.medium.com/unit-testing-with-jest-ab03e7e923b6.
7. **Geeks for Geeks.** Geeks for Geeks. *Testing with Jest.* [Online] 07. 02 2025. [Citace: 22. 05 2025.] https://www.geeksforgeeks.org/testing-with-jest/.
8. **Google.** Google. *Developer Content Policy.* [Online] [Citace: 15. 05 2025.] https://play.google/developer-content-policy/.
9. **Express.** ***Routing.*** [Online] [Citace: 15. 05 2025.] https://expressjs.com/en/guide/routing.
10. **Shankar, Binayak G.** Medium. *Getting started with Prisma ORM.* [Online] 10. 10 2023. [Citace: 14. 05 2025.] https://medium.com/@binayakgourishankar/getting-started-with-prisma-orm-89c2fcda5026.

A. Seznam přiložených souborů

Na přiloženém datovém nosiči se nacházejí následující soubory a složky:

- **RP2024-25_Kocnar_Vetne-cleny.docx** – editovatelná verze dokumentace ročníkové práce
- **RP2024-25_Kocnar_Vetne-cleny.pdf** – tisknutelná verze dokumentace ročníkové práce
- **seznam-pouzitych-promptu.pdf** – seznam použitých promptů pro umělou inteligenci, která vytvořila kus zdrojového kódu.
- **Aplikace** – zdrojové kódy